Chapter 6: Creating ActiveX Controls

ActiveX controls offer you an ideal way to create reusable code. Additionally, ActiveX controls inherit and support ActiveX functionality, such as in-place activation, Automation, and drag-and-drop editing.

This chapter focuses on the MFC implementation of ActiveX controls. You will learn how to implement ActiveX controls by using MFC ActiveX ControlWizard.

Objectives

After completing this chapter, you will be able to:

- Describe the advantages of ActiveX control technology.
- Describe the elements of an ActiveX control.
- Explain the features of ControlWizard in creating an ActiveX control.
- Describe the primary tasks of an ActiveX control container.
- Explain the interaction between an ActiveX control container and an ActiveX control.
- Use ControlWizard to create skeletal code for your ActiveX control.
- Use ClassWizard to define properties, methods, and events for your ActiveX control.

165 Overview of ActiveX Controls

This section explains how to create ActiveX controls by using the MFC ActiveX ControlWizard. Using ControlWizard, you can create up to 99 separate controls per project and set individual options for each control.

This section includes the following topics:

- Advantages of ActiveX Controls
- Using the MFC ActiveX ControlWizard
- Registering an ActiveX Control
- Painting an ActiveX Control

Notes

166 Advantages of ActiveX Controls

ActiveX technology makes it easy to develop platform-independent components. ActiveX controls are components that you can plug into many different types of applications called ActiveX control containers. Microsoft Access, Visual Basic, Visual C++, and HTML Web pages are examples of ActiveX control containers. The more you use ActiveX controls, the easier it is to develop and implement applications.

Benefits for Developers

ActiveX controls offer several benefits to developers:

- Your productivity can increase because you develop the code for a feature only once.
- Once an ActiveX control has been developed, support for it is immediately provided through any ActiveX control container, with no customization required.
- You can add standard or custom ActiveX controls to HTML Web pages. ActiveX controls enable the Web user to interact with the page or enter data on the page, rather than just view its contents.

Benefits for Users

As the user of a an ActiveX control, you gain the following benefits:

- You can select from a wide range of advanced features and quickly add them to your applications without having to develop the idea or the code for the features from the ground up.
- You can easily use the expertise and efforts of others to improve your applications.

Notes

167 Using the MFC ActiveX ControlWizard

The MFC ActiveX ControlWizard is provided with Microsoft Visual C++ 6.0. ControlWizard creates the skeleton code and the necessary project files for an ActiveX control and implements the code for selected options.

▶ **To launch ControlWizard**

1. On the **File** menu, click **New**, and then click the **Projects** tab.
2. Enter a name for your project and double-click **MFC ActiveXControlWizard** to start creating your control.

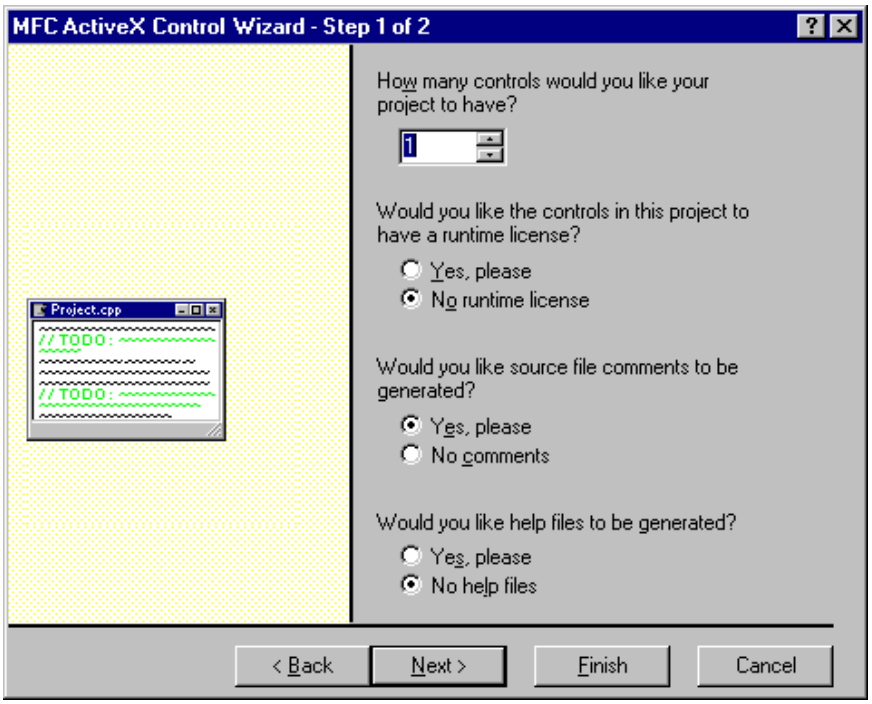
To see a demonstration that shows how to use ControlWizard to create ActiveX controls, click this icon.  (CD-ROM plays the demonstration, "Using the MFC ActiveX ControlWizard.")

▶ **To complete Step 1 of ControlWizard**

1. Choose the number of controls for your project. Each control has a control class and a property page class.
2. Choose whether you want the controls in your project to have a run-time license.
3. Choose whether you want ControlWizard to generate comments in the source file that will guide you in creating your control.
4. Choose whether you want ControlWizard to generate files for context-sensitive Help. Help support requires that you use the Help compiler, which is provided with Visual C++.
5. Click **Finish** to create the ActiveX control, or proceed to Step 2 by clicking **Next**.

Notes

To see an illustration of the **MFC ActiveX ControlWizard - Step 1 of 2** dialog box, click this icon. (CD-ROM displays the following illustration.)

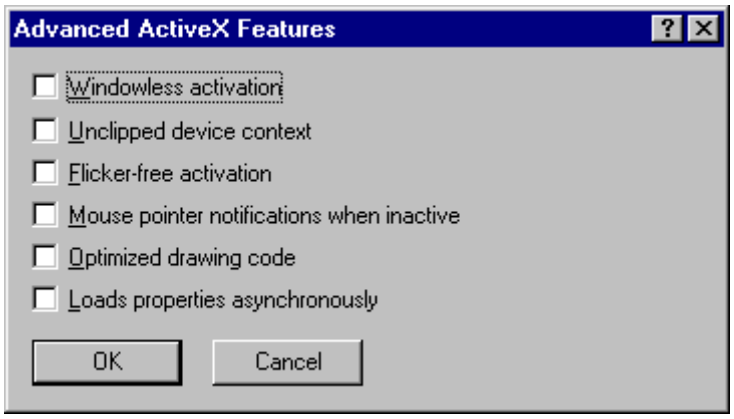


▶ **To complete Step 2 of ControlWizard**

1. If you added more than one control to your project and want to edit the names of the classes or files they use, select the control you want to edit from the list and click **Edit Names**. Repeat this procedure for each control you want to edit.
2. Select the features for each control.
3. Select the window class, if any, that each control should subclass.

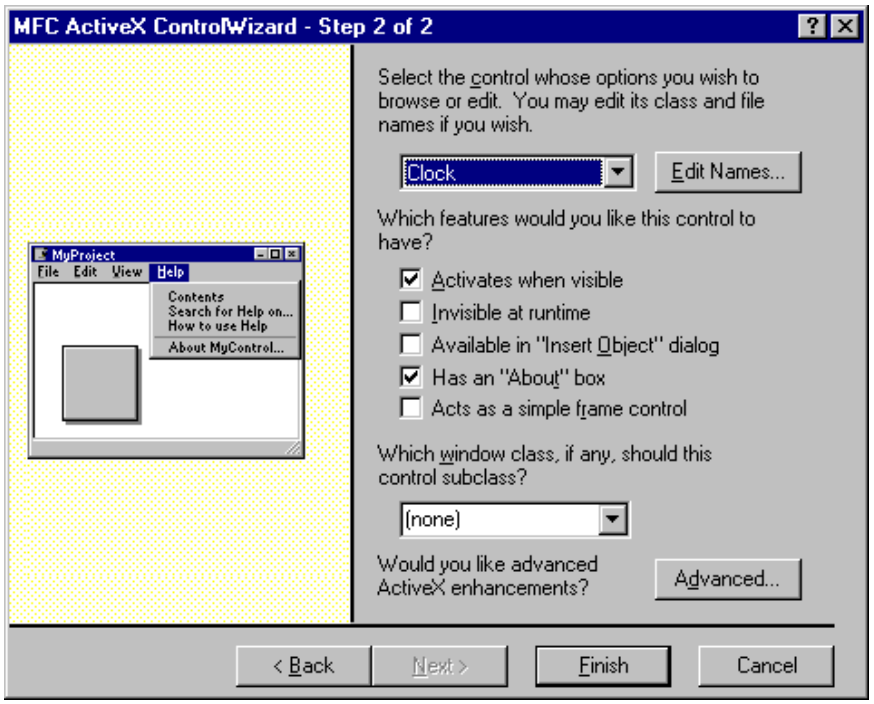
Notes

4. Click **Advanced** to add enhanced ActiveX features to your control. The **ActiveX Advanced Features** dialog box will appear as shown in the following illustration.



5. Select the advanced features for your control and click **OK**.
6. Click **Finish** to create the project files for your control.
7. Click **OK** to place your control in the appropriate folder and prepare it for use.

To see an illustration of the **MFC ActiveX ControlWizard - Step 2 of 2** dialog box, click this icon. (CD-ROM displays the following illustration.)



Notes

What ControlWizard Provides

ControlWizard generates a set of core project files that every control must have, plus any additional files that are required for optional features, such as context-sensitive Help or licensing.

Additionally, ControlWizard generates three main classes for each control, as shown in the following table.

Class	Description
Control module	Used to initialize the ActiveX control DLL. Typically, you do not need to modify this class. The default class name is **CXxxApp**, where **Xxx** is the name of the control.
Control	Provides control-specific behavior, including drawing and user interaction code. The default class name is **CXxxCtrl**, where **Xxx** is the name of the control.
Property page	Default property page for the control. Generally available at application design time, the property page(s) enables the developer to expediently initialize the properties of a control. The default class name is **CXxxPropPage**, where **Xxx** is the name of the control.

What MFC Provides

MFC provides the **COleControl** class, a powerful base class that encapsulates much of the required implementation for an ActiveX control. Derived from **CWnd**, this class inherits all the functionality of a Windows window object, plus additional functionality specific to ActiveX, such as event firing and the ability to support methods and properties.

Notes

169 Registering an ActiveX Control

One requirement for success in using component technology is having a single, searchable location that contains information about available components and their capabilities. This location is an operating system database known as the registry, which is available in Windows 95 and Windows NT.

For a control to be available for general use, it must register itself in the HKEY_CLASSES_ROOT section of the registry. When your ActiveX control is created with ControlWizard, the code that is necessary to accomplish registration is provided in the control module class. The control automatically registers itself every time it is loaded into memory. Registering the control automatically every time it is loaded minimizes the possible damage if a user mistakenly corrupts the registry database.

 (CD-ROM displays the following tip.)

The HKEY_CLASSES_ROOT section of the registry lists various items, including the name of the various OLE controls. A CLSID entry appears as a subkey under each item. Selecting this entry displays a large number consisting of 128-bit hexadecimal numbers. These numbers are known as globally unique identifiers (GUIDs) and are statically guaranteed to be unique when generated randomly. Specifically, they are known as class IDs, or CLSIDs, and uniquely identify each control and property page. Each class ID maps to an entry under the HKEY_CLASSES_ROOT\CLSID key, which completely defines the control information that is stored in the registry, including: its location, whether the control appears in the **Insert Object** dialog box, and so on.

You can register any DLL function by adding a shortcut to RegSvr32.exe in the \Windows\Sendto folder. After you add the shortcut, right-click the ActiveX control's .ocx file, click **Send To** on the shortcut menu, and then click **Shortcut to RegSvr32.exe**. The ActiveX control will be registered.

Notes

170 Painting an ActiveX Control

Some controls provide meaningful functionality but require no visible user interface. However, many of the controls require painting code to display information to the user.

The OnDraw Function

The framework provides an **OnDraw** function in your control class, complete with an appropriate device context (DC). The header file for the **COleControl::OnDraw** function contains the following arguments:

OnDraw(**CDC*** pDC**, const CRect&** rcBounds, **const CRect&** rcInvalid)

The pDC argument contains a DC in which the drawing occurs. The rcBounds argument defines the rectangular area of the control, including the border. The rcInvalid argument defines the rectangular area of the control that is invalid.

The **OnDraw** function is called whenever the framework or control container determines that the control is dirty and needs repainting. However, not only does the control display its information, it must also blend with the control container; that is, it should appear to be part of the container.

To blend with a container, an ActiveX control can query the control container for ambient properties such as background color, text color, and current font, and set its own properties to match those of the container.

MFC provides a function called **COleControl::InvalidateControl** that is analogous to the **CWnd::Invalidate** function. **InvalidateControl** forces immediate repainting of either the whole control window or just the rectangle specified in the function call. **InvalidateControl** is frequently called in functions that are used to set visual properties of the control.

Using OnDraw at Run Time vs. Design Time

The **OnDraw** function of an ActiveX control is called under two different circumstances: at run time and at design time.

Run Time

When the application that contains the control is running, **OnDraw** is invoked in a manner similar to an MFC-based application's view class. OnDraw: WM_PAINT messages are routed to **COleControl::OnPaint**, which creates a DC and then calls **OnDraw**. When a control is painted during run time, it is considered active.

Design Time

When a control is being used in a design environment, such as Visual Basic or the Visual C++ dialog editor, the application in which the control will eventually be used is not running; however, the control still must be drawn on the screen. Your control's **OnDraw** function is used, but the mechanism that calls **OnDraw** is different. When a control is painted at design time, it is considered inactive.

When the control is inactive, its window is invisible or nonexistent, so it cannot receive a paint message. Instead, the control container directly calls the **OnDraw** function of the control, passing the DC of the control container and the coordinates of the rectangular area occupied by the control. This differs from an active control's painting process in that the **COleControl::OnPaint** member function is never called.

Notes

Modifying the OnDraw Function

Whether the control is active or inactive, the framework calls the **OnDraw** member function. You add the majority of your painting code to this member function.

The rectangle passed by the framework to the **OnDraw** member function contains the area occupied by the control. If the control is active, the upper-left corner is (0, 0) and the DC passed is for the child window that contains the control. If the control is inactive, the upper-left corner is not necessarily (0, 0) and the DC passed is for the control container.

When you modify **OnDraw**, remember that the rectangle's upper-left corner may not necessarily be (0, 0), and that you draw only inside the rectangle passed to **OnDraw**. Unexpected results can occur if you draw outside the defined rectangle area.

Note When painting a control, do not make assumptions about the state of the DC passed as the pDC parameter to the **OnDraw** function. Occasionally, the DC is supplied by the container application and will not necessarily be initialized to the default state. In particular, explicitly select the pens, brushes, colors, fonts, and other resources on which your drawing code depends.

Notes

171 ActiveX Control Properties

Control properties, as opposed to ambient properties, are properties that the control exposes to its container. This section describes the two types of control properties: stock properties and custom properties.

This section explains how to use ClassWizard to implement control properties. You will also learn how to access ambient properties and how to set and return control properties at run time.

This section includes the following topics:

- Stock Properties
- Custom Properties
- Using Get/Set Methods to Implement Control Properties
- Parameterized Properties
- Accessing Ambient Properties

Notes

172 Stock Properties

Properties that are typically supported by all ActiveX controls are referred to as stock properties. ControlWizard provides standard implementations of stock properties to make developing controls easier. The following table describes some common stock properties.

Property	Description
BackColor	Determines the color used to paint the background of the control.
BorderStyle	Determines what sort of border the control should have.
FillColor	Determines the color used to fill in shapes.
Font	Determines the font the control uses for text output.
Enabled	Determines whether the control is enabled.
Text or **Caption**	Determines the text/caption assigned to the control. (These properties represent the same characteristic.)

 (CD-ROM displays the following tip.)

For a complete list of stock properties, see the Olectl.h file. This file can be found in the Include subfolder of Microsoft Visual Studio.

Adding Stock Properties

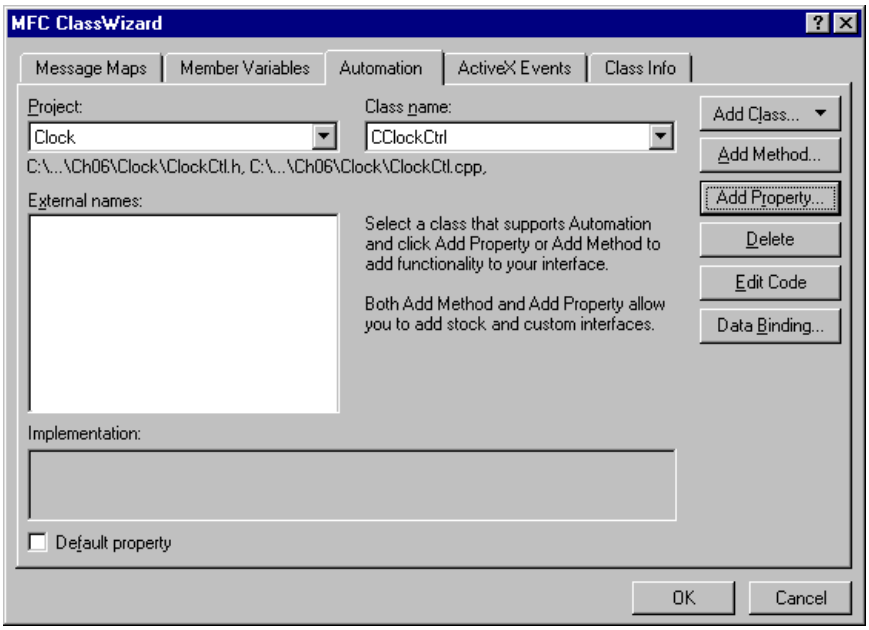
Although standard implementations of stock properties are provided, you are not required to support them. Use ClassWizard to add only those properties that your ActiveX control will support.

▶ **To add a stock property to an ActiveX control**

1. Open the project file and start ClassWizard.
2. On the **Automation** tab, select the control class and click **Add Property**.
3. Select the property name from the **External name** box, and then click **OK**. The names enumerated in the **External name** box are the stock properties.

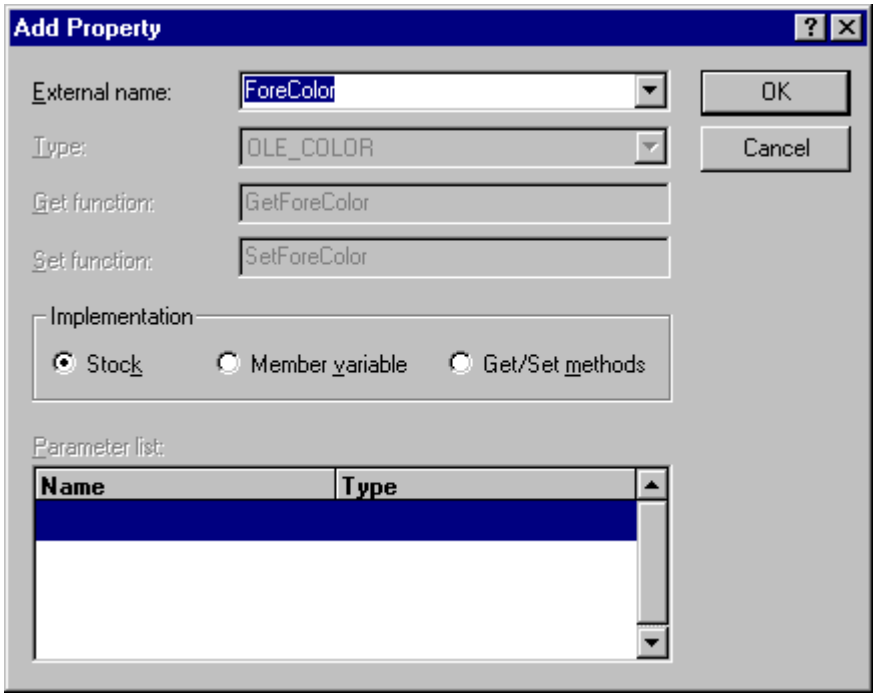
Notes

To see an illustration of the **Automation** tab in the **MFC ClassWizard** dialog box, click this icon. (CD-ROM displays the following illustration.)



Notes

To see an illustration of the **Add Property** dialog box, click this icon.
 (CD-ROM displays the following illustration.)



What ClassWizard Provides

For each stock property that you include, ClassWizard adds a **DISP_STOCKPROP_xxx** entry to the dispatch map of the control, where **xxx** is the property name. For example, **DISP_STOCKPROP_FORECOLOR** is added to provide support for the **ForeColor** stock property, as shown in the following example:

```
BEGIN_DISPATCH_MAP(CClockCtrl, COleControl)
    //{{AFX_DISPATCH_MAP(CClockCtrl)
    DISP_STOCKPROP_FORECOLOR()
    //}}AFX_DISPATCH_MAP
END_DISPATCH_MAP()
```

Notes

When a stock property is modified, it may be important for your application to be notified. Most stock properties have an associated notification function that, by default, calls **InvalidateControl** to force the control to repaint. For example, the **BackColor** stock property has an **OnBackColorChanged** notification function that can be overridden by your control class to get explicit notification that the backcolor has changed, as shown in the following example:

```
void CClockCtrl::OnBackColorChanged()
{
    MessageBox("Background Color Changed");
    COleControl::OnBackColorChanged();
}
```

The **BackColor** property is added in very much the same manner as the **ForeColor** property.

Notes

173 Custom Properties

Custom properties are properties you define as you develop an ActiveX control. Through custom properties, your ActiveX control assumes its unique characteristics.

The following table describes the four custom control implementations.

Type	Dispatch map entry macro	When to use
Member variable with notification	DISP_PROPERTY_NOTIFY	Use when you need to be notified that a property value has changed. The controller changes the property directly, but a notification function is called to inform the control. ClassWizard adds the data member.
Member variable	DISP_PROPERTY	Use when it is not important to know when the property value changes. The controller changes the property directly. ClassWizard adds the data member, but you must delete the notification function in the **Add Property** dialog box to set this property.
Get/**Set** methods	DISP_PROPERTY_EX	Use when you need to compute the value of a property at run time, perform validation, or implement read-only or write-only properties. Use the **Get**/**Set** methods to add the variable to the control class.
Parameterized	DISP_PROPERTY_PARAM	Use to access a set of values through a single control property. You must add the data member. This implementation allows **Get**/**Set** methods to add as many arguments to the control as needed to expose a property.

The **Get** and **Set** methods are the most widely used implementation. They give the ActiveX control explicit handling of property interaction by the control container. Typically, the container will not necessarily call the **Get** or **Set** function; the appropriate function will be called depending on the use of the property in the container. For sample code that shows how to use a control's properties in Visual C++, see Using an ActiveX Control in an MFC-based Application in Chapter 5: Using Controls.

Notes

Adding Custom Properties

Adding custom properties to your ActiveX control is similar to adding stock properties. The main difference between the two is that when you add a custom property, you must enter a custom property name and parameters.

▶ **To add a custom property to an ActiveX control**

1. Load your control's project and start ClassWizard.
2. On the **Automation** tab, choose the control's class from the **Class name** box and click **Add Property**.
3. In the **External name** box, type a name for the custom property.
4. On the **Implementation** tab, select **Member Variable** or **Get/Set Methods**.
5. Select a type for the property, and set other options specific to the desired implementation.
6. Choose **OK** to close the **Add Property** dialog box.
7. Choose **OK** to confirm your choices and close ClassWizard.

To see a demonstration that shows how to use ClassWizard to add properties, click this icon.
 (CD-ROM plays the demonstration, "Adding Properties with ClassWizard.")

Notes

174 Using Get/Set Methods to Implement Control Properties

When you implement a control property by using the **Get**/**Set** methods, the following two functions are created for you:

- **Get**

  This function is called when the control user requests the value of the control's property.

- **Set**

  This function is called when the control user requests that the control's property be changed.

The following example sets the alarm hour to a new value:

```
void CClockCtrl::SetAlarmHour(short nNewValue)
{
    if (nNewValue < 0 || nNewValue >= 24)
    {
        ThrowError(CTL_E_INVALIDPROPERTYVALUE, "AlarmHour must be between 0
and 23, inclusive");
    }

    m_AlarmHour = nNewValue;
    SetModifiedFlag();
}
```

The following code sample returns the value of the alarm hour that has been set.

```
short CClockCtrl::GetAlarmHour()
{
    return m_AlarmHour;
}
```

When you select the **Get**/**Set** methods implementation in ClassWizard, you can either accept the default name of these functions, or specify unique names.

Implementing Read-Only and Write-Only Properties

By using ClassWizard, you can easily make a control property read-only or write-only by deleting the **Get** or **Set** function in the **Get**/**Set** methods implementation. To create a read-only property, you delete the **Set** function. To create a write-only property, you delete the **Get** function

When the DISP_PROPERTY function is added to the dispatch map, the function that was cleared will default to the function **SetNotPermitted** or **GetNotPermitted**, as appropriate. These functions simply throw an appropriate error back to the container and notify it that the attempted **Set** or **Get** is not supported for the property.

Notes

175 Parameterized Properties

A parameterized property (also called a property array) enables you to specify a set of values for a single property. When this property is used by the control container application, individual values are accessed using array notation. For sample code showing how to use a parameterized property in an MFC-based application, see Using an ActiveX Control in an MFC-based Application in Chapter 5: Using Controls.

▶ **To add a parameterized property by using ClassWizard**

1. Add a new property using the **Get**/**Set** method.
2. In the **Parameter** list, use the grid control to add parameters and specify the type you want for each. (You can use up to 15 parameters.)

When specifying parameter values that are dynamic, the control will usually expose read-only properties for the row and column parameters. For example, if you build a temperature control using the number of hours in a day (24) as the row parameter and the maximum days in a month (31) as column parameter, the control will expose "hours in day" and "days in month" as read-only properties. If the control container passes invalid values, the control will generally use the function **ThrowError** to notify the container of the problem.

Notes

176 Accessing Ambient Properties

Often, you will want a visible ActiveX control to blend into the control container application's display. To accomplish this, the control must be able to determine some of the visual properties used by the control container. For example, determining the background color used by the container allows the control to set its own background property to match that of the container. The properties exposed by the control container to a control are known as ambient properties.

Some ambient properties allow the control to determine environmental variables for the control container, such as design mode or run mode in Visual Basic. Based on the mode, the control may behave in a particular manner.

MFC provides a single function call, **COleControl::GetAmbientProperty,** to access any ambient property by specifying its dispatch ID. Some of the more commonly used ambient properties have dedicated functions available, as listed in the following table.

COleControl function	Description
AmbientBackColor	The container's backcolor.
AmbientDisplayName	The container's name for the control.
AmbientFont	The ambient font; note that the caller must call the **Release** function on the returned font.
AmbientForeColor	The container's forecolor.
AmbientLocaleID	The locale identifier that allows the control to localize itself.
AmbientScaleUnits	Displays positions or dimensions, labeled with the chosen unit, such as twips or centimeters.
AmbientTextAlign	The container's text alignment.
AmbientUserMode	Used by containers that support two modes of operation; for example, design mode or run mode in Visual C++.
AmbientUIDead	Allows the control to determine whether the container wants it to react to user-interface operations.
AmbientShowHatching	Determines whether the container allows the control to display itself with a hatched pattern when the user interface is active.
AmbientShowGrabHandles	Determines whether the container allows the control to display grab handles for itself when active.

Notes

 (CD-ROM displays the following tip.)

For a complete list of the dispatch IDs for the standard set of ambient properties, see the Olectl.h file. This file can be found in the Include subfolder of Microsoft Visual Studio.

A control container does not need to provide information about the ambient properties. In this case, you can use a reasonable default value for any ambient property values not supplied.

Notes

177 ActiveX Control Methods

Methods enable a control container to request that an ActiveX control perform some action.

In this section, you will learn how to use ClassWizard to add ActiveX control methods. You will also learn about the stock methods available for use by your control, and how to add custom methods. Finally, this section describes the syntax used by the control container application when ActiveX control methods are invoked.

This section includes the following topics:

- Adding Stock Methods
- Adding Custom Methods

Notes

178 Adding Stock Methods

ClassWizard provides implementations for the stock methods **DoClick** and **Refresh** as follows.

Method	Description
DoClick	Simulates the action of clicking the left mouse button. This method causes the **COleControl** member function **OnClick** to be called and causes the **Click** event to be called (if supported).
Refresh	Calls **COleControl::Refresh**, which causes the control to be repainted.

You are not required to support these methods, but most ActiveX controls support them.

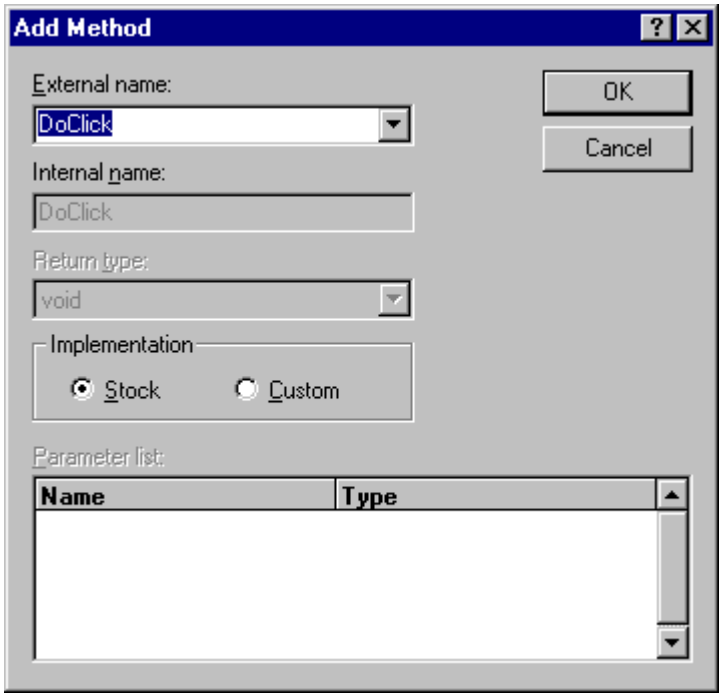
You use ClassWizard to add stock methods to an ActiveX control much as you would to add properties.

▶ **To add a stock method to an ActiveX control**

1. In the **MFC ClassWizard** dialog box, click the **Automation** tab.
2. Choose the control's class from the **Class name** box, and click **Add Method**.
3. In the **External name** box, select a stock method name from the drop-down list, then click **OK** to close the **Add Method** dialog box.
4. Click **OK** to confirm your choices and close ClassWizard.

Notes

To see an illustration of the **Add Method** dialog box, click this icon.
(CD-ROM displays the following illustration.)



As with stock properties, no additional code is added to your application, except that an appropriate entry is inserted into the control's dispatch map. The code necessary to handle the method is part of the base class.

Notes

179 Adding Custom Methods

Use custom methods to define control-specific actions for your ActiveX control. Custom methods typically perform a specific task. For example, the **Clock** control uses an **AlarmTest** method for testing the alarm setting.

In ClassWizard, you can create a custom method, and define a return value type (if desired) and the parameters that will be passed to the method.

▶ **To add a custom method to an ActiveX control**

1. Load your control's project.
2. In the **MFC ClassWizard** dialog box, on the **Automation** tab, choose the control's class from the **Class name** box and click **Add Method**.
3. In the **External name** box, type a name for the custom method. (By not selecting a method from the drop-down box, you are defining a custom method.)
4. In the **Internal name** box, type the name of the method's internal function, or use the default value provided by ClassWizard.
5. In the **Return Type** box, select a data type.
6. In the **Parameter** list, add any parameters you want and select a data type for each parameter.
7. Click **OK** to close the **Add Method** dialog box.
8. Click **OK** to confirm your choices and close ClassWizard.

As when adding a stock method, an appropriate entry is added to the dispatch map of the control. Additionally, a skeletal function is provided in the control class; you must update this function to provide the behavior you want for the method.

To see a demonstration that shows how to use ClassWizard to add custom methods to an ActiveX control, click this icon.

 (CD-ROM plays the demonstration, "Adding Methods with ClassWizard.")

Notes

180 ActiveX Control Events

Properties and methods enable the control container to request or submit information and ask the ActiveX control to perform actions. Events provide a way for the ActiveX control to communicate with the control container, without requiring an explicit request from the container. Events also are important when communicating errors to the container.

An event can take up to 15 parameters, which enable it to pass information about itself to the control container. However, you cannot return results to the ActiveX control through event invocations.

This section describes how to add stock and custom events to a control by using ClassWizard.

This section includes the following topics:

- Adding Stock Events
- Adding Custom Events

Notes

181 Adding Stock Events

Stock events are events that the **COleControl** class handles automatically. It is not required that you support all these events, in your ActiveX control. The following table describes some of the stock events that ClassWizard can implement for you.

Event	Event fires when:
Click	The user clicks the control.
DblClick	The user double-clicks the control.
Error	The control wants to notify its container that an error has occurred.
KeyDown	The user presses a key while the control has focus.
KeyUp	The user releases a key while the control has focus.
MouseDown	The user clicks a mouse button over the control.
MouseMove	The user moves the mouse over the control.
MouseUp	The user releases a mouse button over the control.

▶ **To add a stock event to an ActiveX control**

1. Load your control's project, and start ClassWizard.
2. On the **ActiveX Events** tab, select the control's class from the **Class name** box.
3. Click **Add Event**, and select one of the stock events from the **External name** box.
4. Click **OK** to close the **Add Method** dialog box.
5. Click **OK** to confirm your choices and close ClassWizard.

ClassWizard adds an entry in the event map section of the control's implementation file. For example, if the **Click** event is added to the **Clock** control, ClassWizard adds the following code to the event map section of the file ClockCtrl.cpp:

```
BEGIN_EVENT_MAP(CClockCtrl, COleControl)
    //{{AFX_EVENT_MAP(CClockCtrl)
    EVENT_STOCK_CLICK()
    //}}AFX_EVENT_MAP
END_EVENT_MAP()
```

Notes

182 Adding Custom Events

Using custom events, you can define unique ways for your control to notify its container that something has occurred. A custom event can be based on events that are visual in nature and occur as a result of some user action, such as clicking a particular portion of the control. Custom events can also be based on other occurrences, such as the passing of time or a ringing telephone line.

Stock events are fired by **COleControl** at the appropriate times. Custom events must be fired explicitly by the derived control class. For example, when an alarm is signaled in the **Clock** control, the control must explicitly fire the custom alarm event for the control container to receive notification of the event.

Adding Custom Events

As with stock events, you implement custom events by using the **ActiveX Events** tab in ClassWizard.

▶ **To add a custom event to a control**

1. Load your control's project.
2. Start ClassWizard.
3. On the **ActiveX Events** tab, choose the name of the control class from the **Class name** box and click **Add Event**.
4. In the **External name** box, type the name of your event.
5. The **Internal name** box displays the default name for the event firing function, **FireXxxx**, where "Xxxx" is the name of your event. You can accept the default name, or enter a different name.
6. Add parameters you want to the **Parameter** list.
7. Click **OK** to close the **Add Event** dialog box.
8. Click **OK** again to confirm your choices and close ClassWizard.

The following occurs when you use ClassWizard to add a custom event:

- A function called **FireXxxx** is created (where "Xxxx" is the name of the custom event). This function calls **COleControl::FireEvent**, passing to it the custom event and any parameters that have been defined.
- An entry is made to the control's event map that maps the custom event to the **FireXxxx** function.
- A line of code is added to the control's .odl file that assigns the custom event a specific ID number. This entry allows the control container to anticipate the event. For example, if the control is used in a Visual Basic-based application, the event will be listed in the code window's procedure list box.

Once the custom event is in place, you can call the **FireXxxx** function at any point in your code where that event has occurred.

To see a demonstration that shows how to use ClassWizard to add custom events to an ActiveX control, click this icon

 (CD-ROM plays the demonstration, "Adding Events with ClassWizard.")

Notes

187 Implementing ActiveX Control Property Pages

This section describes how to implement ActiveX control property pages. It discusses default, stock, and custom property pages, and also explains how to transfer data between controls and property pages. A sample **Clock** control is used to show how to implement the code to modify and set the properties for a control.

This section includes the following topics:

- Introduction to ActiveX Control Property Pages
- Implementing the Default Property Page
- Exchanging Data Between Controls and Property Pages
- Adding Stock Property Pages
- Adding Custom Property Pages

Notes

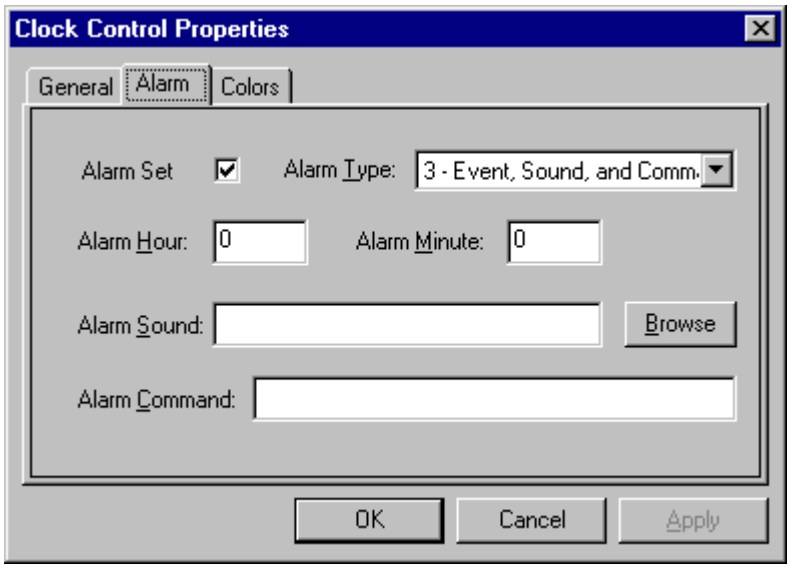
188 Introduction to ActiveX Control Property Pages

A control container supports access to a control's properties at design time. However, different containers provide different implementations to set design-time properties. For example, Visual Basic provides a Properties window that may not be available in other containers. To ensure that developers can set properties at design time regardless of the type of control container your ActiveX control is used in, you should implement a container-independent method to set properties.

With ActiveX control property pages, you can display and alter a control's properties without affecting the control container displaying the control. ActiveX control property pages will work with any type of container control.

Using property pages also enables you to provide a dialog-type interface for setting properties.

To see an illustration of an example of a property page for an ActiveX control, click this icon. (CD-ROM displays the following illustration.)



Notes

189 Implementing the Default Property Page

When you create an ActiveX control by using ControlWizard, the template provides for one default property page. This includes both an empty property page resource and the code to invoke the property page.

A property page appears as a tabbed page on a dialog box that acts very much like a standard dialog box. You can add controls to the default property page, and add more pages to suit the needs of your ActiveX control.

To view the default property page for any ActiveX control, right-click the control and select **Properties** on the context menu.

To add controls to a property page, you follow nearly the same set of steps as you would to add a control to a dialog box.

▶ **To add a control to a property page**

1. In the Dialog section in ResourceView, open the appropriate property page template.
2. Add a control to the template that will be associated with a control property.
3. Start ClassWizard to add a member variable to hold the property value.
4. Click the **Member Variables** tab.
5. In the **Control IDs** list, select the control for which the member variable is to be added and click **Add Variable**.
6. Enter the member variable name in the **Add Member Variable** dialog box.
7. Set the category to **value** and ensure that the variable type is accurate.
8. In the **Optional Property Name** combo box, enter the name of the ActiveX control property associated with this variable, or choose a stock property, and click **OK**.

As it does when creating dialog boxes, ClassWizard performs several tasks in this situation. First, it declares and initializes member variables. Second, it adds an appropriate entry to the **DoDataExchange** function to transfer information between the member variables and the controls, as appropriate. However, **DoDataExchange** takes on an additional task when implementing default property pages, as described in Exchanging Data Between Controls and Property Pages.

Notes

190 Exchanging Data Between Controls and Property Pages

DoDataExchange is designed to transfer data between controls on the property page and the member variables in the property page class to link property page values with the actual values of properties in the ActiveX control. The DDX group of functions handles these transfers. You must map the appropriate property page fields to their respective control properties to establish links. This enables the property page object to get and set control properties, as an ActiveX container would.

You use the property page DDP functions to map property page fields to control properties. The DDP functions work much like the DDX functions used in standard MFC dialog boxes, with one exception: In addition to the reference to a member variable, DDP functions take the name of the control property. The following is a typical entry in the **DoDataExchange** function for a property page:

```
DDP_Text(pDX, IDC_CAPTION, m_caption, _T("Caption"));
```

This function associates the property page's *m_caption* member variable with the **Caption** property of the ActiveX control.

The following table describes the DDP functions available in MFC.

Function name	Purpose
DDP_CBIndex	Links the selected string's index in a combo box with a control property.
DDP_CBString	Links the selected string in a combo box with a control property. The selected string can begin with the same letters as the property's value, but need not match it fully.
DDP_CBStringExact	Links the selected string in a combo box with a control property. The selected string and the property's string value must match exactly.
DDP_Check	Links a check box with a control property.
DDP_LBIndex	Links the selected string's index in a list box with a control property.
DDP_LBString	Links the selected string in a list box with a control property. The selected string can begin with the same letters as the property's value, but need not match it exactly.
DDP_LBStringExact	Links the selected string in a list box with a control property. The selected string and the property's string value must match exactly.
DDP_Radio	Links a radio button with a control property.
DDP_Text	Links text with a control property.

Notes

To see sample code for the **DoDataExchange** function, click this icon.

 (CD-ROM displays the following sample code.)

```
//Transfers data between controls on the property page
//and the member variables in the property page class
//Samples/CH06/Clock

void CAlarmPropPage::DoDataExchange(CDataExchange* pDX)
{
    // NOTE: ClassWizard will add DDP, DDX, and DDV calls here
    //    DO NOT EDIT what you see in these blocks of generated code !
    //{{AFX_DATA_MAP(CAlarmPropPage)
    DDP_Text(pDX, IDC_ALARM_HOUR, m_AlarmHour, _T("AlarmHour") );
    DDX_Text(pDX, IDC_ALARM_HOUR, m_AlarmHour);
    DDV_MinMaxInt(pDX, m_AlarmHour, 0, 23);
    DDP_Text(pDX, IDC_ALARM_MINUTE, m_AlarmMinute, _T("AlarmMinute") );
    DDX_Text(pDX, IDC_ALARM_MINUTE, m_AlarmMinute);
    DDV_MinMaxInt(pDX, m_AlarmMinute, 0, 59);
    DDP_Text(pDX, IDC_ALARM_SOUND, m_AlarmSound, _T("AlarmSound") );
    DDX_Text(pDX, IDC_ALARM_SOUND, m_AlarmSound);
    DDP_Text(pDX,IDC_ALARM_COMMAND,m_AlarmCommand,_T("AlarmCommand"));
    DDX_Text(pDX, IDC_ALARM_COMMAND, m_AlarmCommand);
    DDP_Check(pDX, IDC_ALARM_SET, m_bAlarmSet, _T("AlarmSet") );
    DDX_Check(pDX, IDC_ALARM_SET, m_bAlarmSet);
    DDP_CBIndex(pDX, IDC_ALARM_TYPE, m_AlarmType, _T("AlarmType") );
    DDX_CBIndex(pDX, IDC_ALARM_TYPE, m_AlarmType);
    //}}AFX_DATA_MAP
    DDP_PostProcessing(pDX);
}
```

Your property page's **DoDataExchange** function includes a call to **DDP_Processing** to complete the transfer of data to the control.

Notes

191 Adding Stock Property Pages

Beginning with Windows 3.1, Microsoft has provided implementation for a set of commonly used dialog boxes, such as those used when you open and save files. Use of these common dialog boxes eliminates a significant amount of work that was previously required of programmers, and provides a consistent interface to the user across all applications. Similarly, MFC provides implementation for several commonly used property pages that you can use in your application. These stock property pages include the font chooser, the color chooser, and the picture chooser.

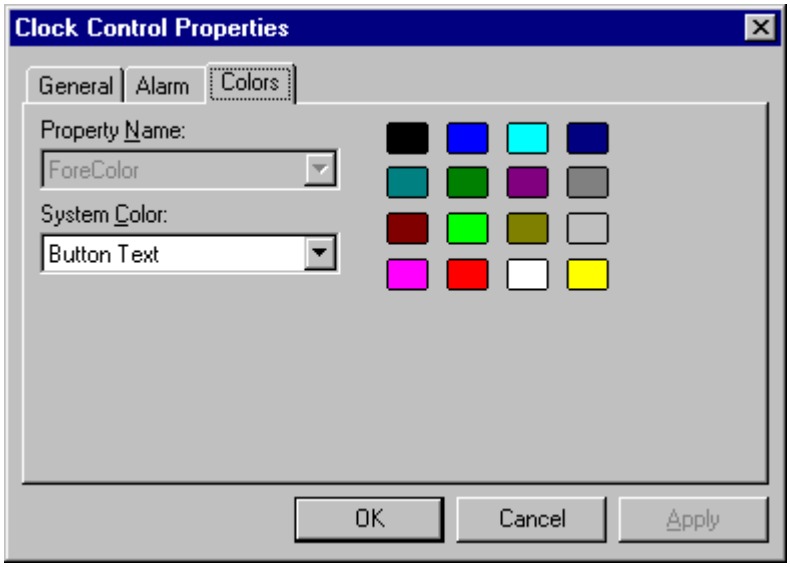
To invoke a stock property page, update the control's property page macro set in the property page map by:

- Adding a PROPPAGEID entry with the appropriate class ID for the stock property page, as shown in the following table.

Stock property page	Class ID
Font chooser	**CLSID_CFontPropPage**
Picture chooser	**CLSID_CPicturePropPage**
Color chooser	**CLSID_CColorPropPage**

- Incrementing the number of property pages in the BEGIN_PROPPAGEIDS macro.

The following illustration shows the color chooser stock property page.



Notes

The following sample code contains a property page macro set that uses the color chooser property page for a sample **Clock** control. The complete code for the **Clock** control is located in \Samples\Ch06\Clock.

```
BEGIN_PROPPAGEIDS(CClockCtrl, 3)
    PROPPAGEID(CClockPropPage::guid)
    PROPPAGEID(CAlarmPropPage::guid)
    PROPPAGEID(CLSID_CColorPropPage)
END_PROPPAGEIDS(CClockCtrl)
```

For the changes to the property pages to be enabled, your ActiveX control will need to be reregistered. Beginning with Visual C++ 4.0, this is done by default when the project is rebuilt; otherwise, use RegSvr32.exe.

Notes

192 Adding Custom Property Pages

In addition to stock property pages, you can also add custom property pages to your ActiveX control. As you create additional property pages for your ActiveX control, consider the following suggestions:

- Keep the number of pages to a minimum.
- Try to minimize the complexity of each property page and keep it uncluttered.
- Keep related information together on the same page.

Adding additional property pages requires four main steps:

1. Create the new property page.
2. Add controls to the property page.
3. Create the new property page class.
4. Add the property page to the control's property page list.

To see a demonstration that shows how to add custom property pages to an ActiveX control, click this icon.
 (CD-ROM plays the demonstration, "Adding Custom Property Pages.")

The following steps show how to create the new property page without the ActiveX Control Property Page component in the Components and Controls Gallery. A sample application for a **Clock** control is used to show how the completed code should look.

▶ **To create the dialog resource for the property page**

1. Use the Resource editor to create a new dialog resource.
2. Name the dialog box IDD_PROPPAGE_Xxxx (where "Xxxx" is a name that describes the topic of the property page).
3. Set the dialog box style to create a child window with a thin border.
4. Make sure the **Titlebar** and **Visible** options are cleared.
5. Size the dialog box to be 250 x 62 or 250 x 110 dialog units.

   Although not mandatory, this step will eliminate compiler warnings that the property page is a nonstandard size.

Note If you create your property page using the IDD_OLE_PROPPAGE_SMALL or IDD_OLE_PROPPAGE_LARGE templates, the resource editor will set the size and properties appropriately.

▶ **To create the new property page class**

1. Start ClassWizard and select **Create a New Class**.
2. Provide the name for the new class and ensure that **COlePropertyPage** is the base class.
3. Click **OK**.

Notes

▶ **To associate a property page name and caption**

1. Use the string table editor to add an entry for the property page name. For example, the ID could be IDS_ALARM_PPG and the string "Alarm Property Page."
2. Use the string table editor to add an entry for the property page caption that will be displayed on the page's tab. For example, use IDS_ALARM_PPG_CAPTION and the string "Alarm."
3. Use the property page name to register the property page. In the **UpdateRegistry** function in your property page class, update the **AfxOleRegisterPropertyPage** function to use your property page name ID as the last parameter. This will update the registry so that the property page can be properly invoked when requested. The following example updates the registry for the **Clock** control:

```
BOOL CAlarmPropPage::CAlarmPropPageFactory::UpdateRegistry(BOOL bRegister)
{
    if (bRegister)
        return
AfxOleRegisterPropertyPageClass(AfxGetInstanceHandle(),m_clsid,
IDS_ALARM_PPG);
    else
        return AfxOleUnregisterClass(m_clsid, NULL);
}
```

4. Enable the caption by modifying the property page constructor to pass the caption ID to the base class, **COlePropertyPage**. The following example shows how to accomplish this:

```
CAlarmPropPage::CAlarmPropPage() : COlePropertyPage(IDD,
IDS_ALARM_PPG_CAPTION)
{
    //{{AFX_DATA_INIT(CAlarmPropPage)
    m_AlarmHour = 0;
    m_AlarmMinute = 0;
    m_bAlarmSet = -1;
    m_AlarmSound = _T("");
    m_AlarmCommand = _T("");
    //}}AFX_DATA_INIT
}
```

▶ **To add the property page to the control's property page list**

1. Update the macro block in the control class's property page to use the new property page.
2. Add a new PROPPAGEID entry for the new property page.
3. Increment the number of property pages in the BEGIN_PROPPAGEIDS macro.

   The following example shows how to add the custom Alarm page to the **Clock** control.

```
BEGIN_PROPPAGEIDS(CClockCtrl, 2)
    PROPPAGEID(CClockPropPage::guid)
    PROPPAGEID(CAlarmPropPage::guid)
END_PROPPAGEIDS(CClockCtrl)
```

Notes

You can display the property page at run time by using the **COleControl::OnProperties** function as shown in the following example:

```
void CClockCtrl::OnRButtonUp(UINT nFlags, CPoint point)
{
    COleControl::OnRButtonUp(nFlags, point); // Call the base version.
    COleControl::OnProperties(NULL, this->GetSafeHwnd(), NULL);
}
```

Notes

183 Creating an Enumerated Property

Sometimes a property has several possible values. In such cases, the property page can easily display the valid selections in an appropriate manner, such as in a drop-down combo box.

A number of properties have their own property browsers, such as the Properties window in Microsoft Visual Basic, which provides an alternate way to view the list of valid property values. MFC provides a mechanism to create these browsers with the necessary information so that they, too, display only the valid selections for a property. To limit the list of valid property values for a given property, you must create an enumerated property.

This section describes how to implement enumerated properties.

This section includes the following topics:

- How Enumerated Properties Work
- Implementing an Enumerated Property

Notes

184 How Enumerated Properties Work

When a control container requests the possible values of a property, the framework asks the control for an array of strings that represents those values by calling **OnGetPredefinedStrings**. The control then supplies an array of strings, which is displayed to the user in the property viewer.

The framework then calls **OnGetPredefinedValue** to request the actual property value for one of the predefined strings. Finally, the framework calls **OnGetDisplayString** to attain a string that represents the current property value. This string is usually one of the strings provided in **OnGetPredefinedStrings**.

For example, the **Clock** sample control has a property called **AlarmType** that can take a discrete set of values, as described in the following table.

Property value	String	Resulting action(s)
0	Event Only	Fires alarm event.
1	Event and Sound	Fires alarm event and plays the alarm sound file.
2	Event and Command	Fires alarm event and runs the alarm's command line.
3	Event, Sound, and Command	Fires alarm event, plays the alarm sound file, and runs the alarm's command line.

Notes

185 Implementing an Enumerated Property

The following steps describe how to create an enumerated control property.

▶ **To create an enumerated control property**

1. If it is not already available, add the property.
2. Override **COleControl::OnGetPredefinedStrings**.
   a. Determine the dispatch identifier (DISPID) of the property.
   b. Fill the **CStringArray** array provided with the valid property strings.
   c. Fill the **CDWordArray** parameter with a value you choose to associate with each string. The value has no particular meaning except that it acts as a tag that will be passed to **OnGetPredefinedValue**, and it is **OnGetDisplayString** that will uniquely define a string.
3. Override **COleControl::OnGetPredefinedValue**.
   a. Validate that the DISPID is one of the enumerated properties.
   b. Given the value that represents a string, return the actual property value.
4. Override **COleControl::OnGetDisplayString**.
   a. Validate that the DISPID is one of the enumerated properties.
   b. Given the current value of the property, usually defined by its member variable, return the display string.

Notes

To see sample code that implements an enumerated property, click this icon.
 (CD-ROM displays the following sample code.)

```
// SAMPLES\Ch06\CLOCK
// ClockCtl.cpp

BOOL CClockCtrl::OnGetPredefinedStrings( DISPID dispid,
                            CStringArray* pStringArray,
                            CDWordArray* pCookieArray )
{
    BOOL bResult = FALSE;

    if(dispidAlarmType == dispid)
    {
    // See if the the dispatch ID is for the AlarmType property
        try
        {
            // Fill in the values in pStringArray and pCookieArray with the
AlarmType info
            // ID_ALARMTYPE_BASE is the offset of the beginning ALARMTYPE
property value
            for (int i = 0; i < 4; i++)
            {
                CString Name;
                Name.LoadString(ID_ALARMTYPE_BASE + i);
                pStringArray->Add(Name);
                pCookieArray->Add(i);
            }

            bResult = TRUE;
        }
        catch(CException* e)
        {
            pStringArray->RemoveAll();
            pCookieArray->RemoveAll();
            e->Delete();
            bResult = FALSE;
        }
    }

    if (!bResult)
        bResult = COleControl::OnGetPredefinedStrings(dispid, pStringArray,
pCookieArray);

    return bResult;
}
```

(Code continues on following page.)

Notes

```
BOOL CClockCtrl::OnGetPredefinedValue( DISPID dispid, DWORD dwCookie, VARIANT
FAR* lpvarOut )
{
  if(dispidAlarmType == dispid)
  {
      VariantClear(lpvarOut);
      V_VT(lpvarOut) = VT_I2;
      V_I2(lpvarOut) = (short)dwCookie;

      return TRUE;
  }

  return COleControl::OnGetPredefinedValue(dispid, dwCookie, lpvarOut);
}

BOOL CClockCtrl::OnGetDisplayString( DISPID dispid, CString& strValue )
{
    if(dispidAlarmType == dispid)
    {
        switch(m_AlarmType)
        {
        case 0:
        case 1:
        case 2:
        case 3:
            strValue.LoadString(ID_ALARMTYPE_BASE + m_AlarmType);
            break;
        }
        return TRUE;
    }

    return COleControl::OnGetDisplayString(dispid, strValue);
}
```

Although not shown, the property strings have been stored in the string table resource.

Notes

194 Data Binding in an ActiveX Control

A data-bound control is an ActiveX control that is "data aware." When you implement a data-bound control, you bind a property of the control to a data source. Doing so specifies what data to display in the control. The data source can be a specific field or column in a database, the value of another control property, a real-time data feed, and so on.

This section includes the following topics:

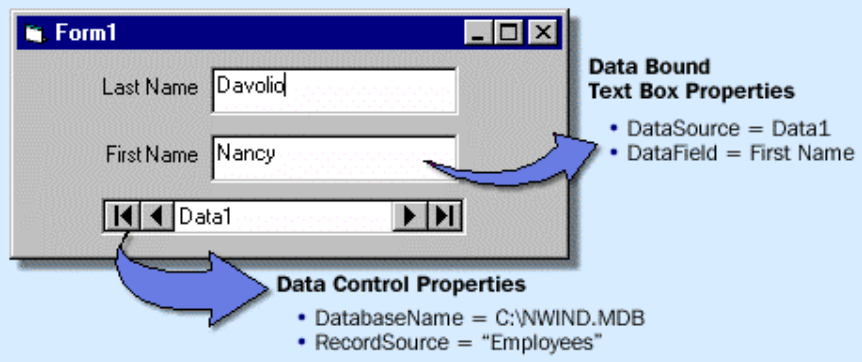
- Introduction to Data Binding
- How Data Binding Works
- Creating a Bindable Property

Notes

195 Introduction to Data Binding

Data binding is a negotiation between a control and a control container, whereby the control requests permission to change some property value, and subsequently informs the container that the data has changed. A common belief is that data binding refers to an ActiveX control that is database aware. Although a data-bound ActiveX control usually is database aware, the mechanism does not enforce this.

In a typical implementation of a data-bound control, controls are placed on a form where they can provide a visual interface to the current record of the database. The following illustration shows a form in Visual Basic that serves as a database front end. The form contains a data control and two data-bound edit boxes to view fields in a database.



The control container must provide the interface between the control and the field in a database. This aspect of data binding is not discussed here. For an example of how a data-bound control is implemented from the control container, search for "DAOCTL: ActiveX Control Container with Data Bound Controls" in MSDN Library Visual Studio 6.0 and see the MFC sample project DAOCTL.

Notes

196 How Data Binding Works

A control with a property designated as bound cannot modify the value of the property without first getting permission from the control container. If permission is given and the value is changed, the control must then notify the control container that the change has occurred.

MFC provides two **COleControl** functions for modifying property values:

- **BoundPropertyRequestEdit**

  Provides the request to the control container to change the data value.
- **BoundPropertyChanged**

  Is called to inform the control container that the data has changed.

Both these functions require a DISPID parameter to identify which property is being addressed.

Each property has associated **Get** and **Set** functions that you must implement. While the **Get** function simply returns the property value as with a nonbound control, the **Set** function must first request permission to edit the property by calling **BoundPropertyRequestEdit**, and inform the control container that the value changed by calling **BoundPropertyChanged**.

Notes

197 Creating a Bindable Property

To create a property that can be bound, select the **Bindable Property** check box in ClassWizard, and provide the appropriate implementation in the property's access functions.

▶ **To create a bindable property**

1. Add a custom property to the ActiveX control using ClassWizard. You must have implemented the property using the **Get**/**Set** methods.
2. On the **Automation** tab in ClassWizard, click **Data Binding**.
3. Select the **Bindable Property** check box.
4. Select any other options that apply, and click **OK** to confirm your choices.
5. On the **Automation** tab, click **Edit Code**.
6. Edit the property's **Set** function.

To see sample code that shows a common implementation of this code, click this icon.

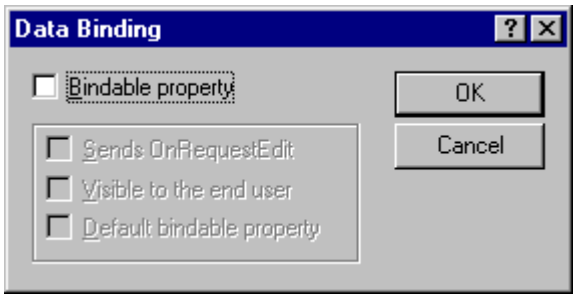
 (CD-ROM displays the following sample code.)

```
//Samples/Ch06/Clock
//Sets the bound property
void CBoundCtrl::SetDataBoundProperty(LPCTSTR lpszNewValue)
{
    // Request permission to change bound property "dispidDataBoundProperty"
    if(BoundPropertyRequestEdit (dispidDataBoundProperty))
    {
        // Change the bound property
        m_strDBProp = lpszNewValue;
        SetModifiedFlag();
        // Notify container that bound property has changed
        BoundPropertyChanged(dispidDataBoundProperty);
        InvalidateControl();
    }
    else
    {
        // Permission to change the property was denied.
        // Use SetNotPermitted to throw an appropriate error to the container.
        SetNotPermitted();
    }
}
```

Notes

Data Binding Options

When you request data binding for a control, ClassWizard presents the following dialog box:



You must select the **Bindable Property** check box for the control to be automatically bound. The remaining options are not required, and are defined in the following table.

Binding option	Description
Sends OnRequestEdit	The property requests permission from the database before modifying the value.
Visible to the End User	The container displays the property in a property-binding dialog box.
Default Bindable Property	Makes the bindable property the control container's default choice.

Notes

199 Optimizing ActiveX Controls

Performance is a crucial factor to consider when creating ActiveX controls. You can use the advanced features of the MFC ActiveX ControlWizard to optimize the performance of controls.

This section includes the following topics:

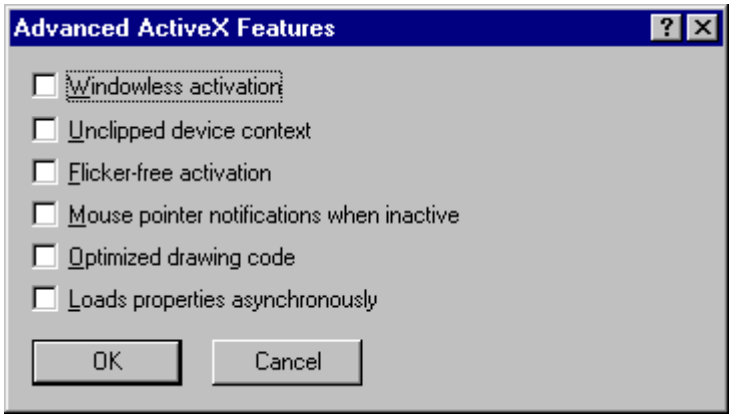
- Advanced Features for ActiveX Controls
- Optimizing Drawing Code
- Windowless Activation

Notes

200 Advanced Features for ActiveX Controls

The MFC ActiveX ControlWizard provides advanced features to help optimize the performance of your ActiveX controls. To select advanced features, click the **Advanced** button in Step 2 of the ActiveX ControlWizard, and then select the features you want in the **Advanced ActiveX Features** dialog box. Choosing any of these features adds a **GetControlFlags** function to the control class.

The following illustration shows the **Advanced ActiveX Features** dialog box.



The base version of **COleControl:: GetControlFlags** returns the value fastBeginPaint /clipPaintDC. The clipPaintDC value causes **COleControl** to make a call to **IntersectClipRect**. If you are certain that your control does not paint outside its client rectangle, you can improve performance by disabling the flag associated with clipPaintDC as follows:

```
DWORD CMyCtrl::GetControlFlags()
{
    return COleControl::GetControlFlags() & ~clipPaintDC;
}
```

The code to remove this flag can be automatically generated for your control. When creating your control with ControlWizard, select the **Unclipped device context** option in the **Advanced ActiveX Features** dialog box. The flag will be removed automatically.

The other flag, fastBeginPaint, is used by **COleControl::OnPaint** (which calls your control's **OnDraw** function) to create a DC optimized for use by controls.

Notes

201 Optimizing Drawing Code

The **OnDraw** function provides optimization through the rcInvalid parameter. This parameter defines the rectangular area of the control that needs redrawing. You can use this area, usually smaller than the entire control area, to speed the painting process.

You can also speed the redrawing of controls by using metafiles. For detailed instructions about how to use metafiles, search for "ActiveX Controls: Painting an ActiveX Control" and "OLE Controls: Top Tips" in the MSDN Library Visual Studio 6.0.

The first parameter of **COleControl**::**OnDraw** is a pointer to a DC. As the control is drawing itself, it will typically create Graphics Device Interface (GDI) objects (such as pens, brushes, and fonts), then select those GDI objects into the DC, perform its drawing operations, and restore the original GDI objects into the DC. This process can take a lot of time and system resources. Furthermore, other controls may be drawing themselves in the container using the same container-supplied DC.

You can improve the performance of a control that uses GDI objects by clicking the **Advanced** button in Step 2 of the ActiveX ControlWizard and selecting the **Optimized drawing code** option. For more information about this option, search for "Optimizing Control Drawing" in the MSDN Library Visual Studio 6.0.

Notes

202 Windowless Activation

The ActiveX Control specification provides windowless activation for ActiveX containers only. Windowless controls are compatible with containers that are not written to support windowless controls. In these containers, the windowless controls simply create a window when active.

When a control maintains an on-screen window, it also has to manage messages for that window. Therefore, windowless controls are faster than windowed controls.

Unlike windowed controls, windowless controls support transparent painting and nonrectangular screen regions. A common example of a transparent control is a text control with a transparent background. The control paints the text but not the background, so whatever is under the text shows through. Newer implementations of this type of control often use nonrectangular controls, such as arrows and round buttons.

For a complete discussion about how to create and use windowless controls, search for "Providing Windowless Activation" in the MSDN Library Visual Studio 6.0.

Notes

203 Debugging and Handling Errors in ActiveX Applications

This section describes methods of communicating errors to ActiveX containers and debugging ActiveX applications.

This section includes the following topics:

- Communicating Errors in ActiveX Controls
- Debugging ActiveX Applications

Notes

204 Communicating Errors in ActiveX Controls

When errors occur in your ActiveX control, you should communicate these errors to the control container. There are two methods of communicating errors:

- **COleControl::ThrowError**

  Call this function if the error occurs within a **Get** or **Set** function, or in the implementation of an Automation method as shown in the following example:

```
void CClockCtrl::SetAlarmMinute(short nNewValue)
{
    if (nNewValue < 0 || nNewValue >= 60)
    {
        ThrowError(CTL_E_INVALIDPROPERTYVALUE,
            "AlarmMinute must be between 0 and 59, inclusive");
    }

    m_AlarmMinute = nNewValue;
    SetModifiedFlag();
}
```

  COleControl provides a set of convenient functions (**SetNotPermitted**, **SetNotSupported**, and **GetNotSupported**) that calls **ThrowError** with appropriate parameters for the type of error.

- **COleControl::FireError**

  Use this function for any other error condition. Calling this function fires the stock **Error** event.

Both functions are easy to use and have parameters that enable you to pass information to the control container about the type of error. For a complete list of ActiveX control error codes, search for “ActiveX Controls: Advanced Topics" in MSDN Library Visual Studio 6.0.

Notes

205 Debugging ActiveX Applications

ActiveX applications perform a number of tasks outside the programmer's direct control. Communication among DLLs, usage counts on objects, and Clipboard operations are just a few areas where you can encounter unexpected behavior. Usually, when unexpected behavior occurs, your first step is to track down the source. Frequently in ActiveX applications, how to debug a particular problem is not immediately obvious.

The MSDN Library Visual Studio 6.0 includes a series of topics explaining techniques that you can use to track down problems, some of which are unique to ActiveX applications. It also introduces you to special tools and testing aids to help you write solid ActiveX applications.

Debugging ActiveX applications begins with the same general debugging techniques you use in other kinds of applications. For more information about debugging ActiveX applications, search for "Debugging ActiveX Applications with MFC" in MSDN Library Visual Studio 6.0.

To debug an ActiveX server and container, start two instances of Visual C++. Load the container into one instance and load the server into the other, set appropriate breakpoints in each, and debug. When the container makes a call into the server that hits a breakpoint, the container will wait until the server code returns. You also can trace calls that exceed process boundaries. For more information on tracing these calls, search for "The Trace Macro" in MSDN Library Visual Studio 6.0.

Notes

207 Sample Applications

The following table describes the sample applications related to this chapter. These sample applications are located in the folder \Samples\Ch06.

Sample application subfolder	Description of application
Clock	A full-featured ActiveX control built using ControlWizard.
UseClock	A **CFormView**-based SDI application that uses the **Clock** control. The **Clock** control must be installed and registered before you run this application.

Notes

208

Self-Check Questions

1. Which dispatch map entry do you use to access a set of values through a single control property?

- A. DISP_PROPERTY_NOTIFY
- B. DISP_PROPERTY
- C. DISP_PROPERTY_PARAM
- D. DISP_PROPERTY_EX

2. Implementing an ActiveX control's default property page most closely resembles which other process?

- A. Adding controls to a normal dialog box
- B. Adding data-bound controls to a **CDaoRecordView** form
- C. Reading and writing to the registry
- D. Associating a toolbar button with a menu item

3. What is a data-bound ActiveX control?

- A. An ActiveX control that is bound to a field in a database
- B. An ActiveX control whose **Get**/**Set** methods have RFX, DFX, or DDX entries to enable the exchange of information
- C. A negotiation between a control and its container, in which the container gives permission to the control to change a property value, and is informed when the control does so
- D. An ActiveX control with information overload

Notes